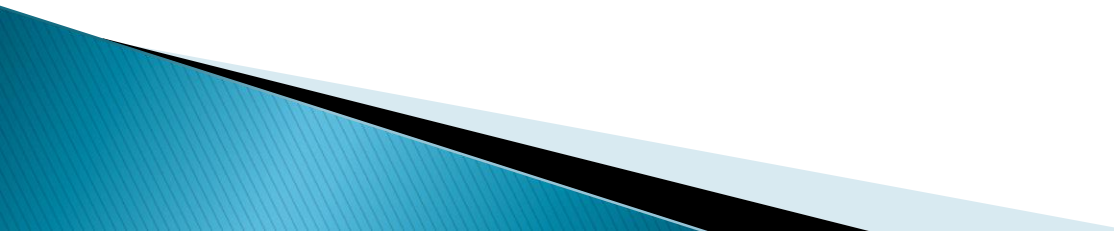


MySQL应用教程

第三章 MySQL数据表操作管理

内容提要

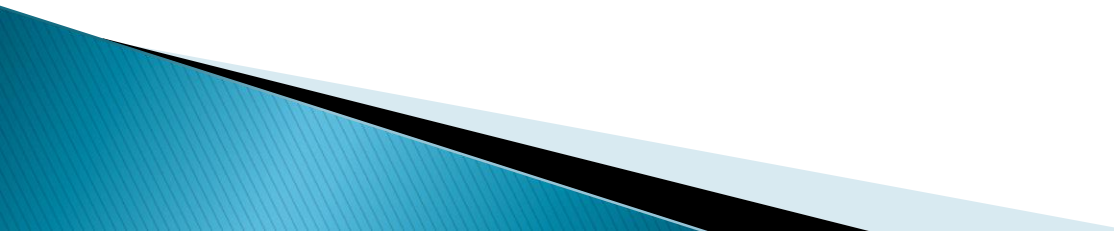
- ▶ 数据表
 - ▶ 创建表
 - ✓ 数据类型
 - ✓ 完整性约束
 - ▶ 查看当前数据库下已有数据表
 - ▶ 查看指定数据表的详细信息
 - ▶ 查看表结构
 - ▶ 删除指定的数据表
 - ▶ 表结构相关操纵
- 

数据表

- ▶ 是数据库最重要的组成部分之一，数据是保存在数据表中
- ▶ 数据表由行(row)和列(column)来组成
- ▶ 每个数据表中至少有一列，行可以有零行一行或者多行组成
- ▶ 表名要求唯一，不要包含特殊字符，最好含义明确

创建表

```
CREATE TABLE [IF NOT EXISTS] tbl_name(  
    字段名称 字段类型 [完整性约束条件][COMMENT 'string'],  
    字段名称 字段类型 [完整性约束条件][COMMENT 'string'],  
    ...  
)[ENGINE=存储引擎 CHARSET=编码方式];
```



数据类型

- ▶ 整型
- ▶ 浮点型
- ▶ 字符型
- ▶ 日期型
- ▶ JSON

整型

数据类型	存储范围	字节
TINYINT	有符号值: -128 到127 (-2^7 到 2^7-1) 无符号值: 0到255 (0 到 2^8-1)	1
SMALLINT	有符号值: -32768 到32767 (-2^{15} 到 $2^{15}-1$) 无符号值: 0到65535 (0 到 $2^{16}-1$)	2
MEDIUMINT	有符号值: -8388608 到8388607 (-2^{23} 到 $2^{23}-1$) 无符号值: 0到16777215 (0 到 $2^{24}-1$)	3
INT	有符号值: -2147683648 到2147683647 (-2^{31} 到 $2^{31}-1$) 无符号值: 0到4294967295 (0 到 $2^{32}-1$)	4
BIGINT	有符号值: -9223372036854775808 到9223373036854775807 (-2^{63} 到 $2^{63}-1$) 无符号值: 0到18446744073709551615 (0到 $2^{64}-1$)	8
BOOL,BOOLEAN	等价于TINYINT(1), 0为false,其余为true	1

浮点型

数据类型	存储范围	字节
FLOAT[(M,D)]	负数取值范围为-3.40E+38到-1.17E-38、0和1.175E-38到3.40E+38。 M是数字总位数，D是小数点后面的位数。如果M和D被省略，根据硬件允许的限制来保存值。单精度浮点数精确到大约7位小数位。	4
DOUBLE[(M,D)]	-1.79E+308到-2.22E-308、0和2.22E-308到 1.79E+308。	8
DECIMAL[(M,D)]	和DOUBLE一样，内部以字符串形式存储数值	M+2

字符型

列类型	存储需求
CHAR(M)	M个字节, $0 \leq M \leq 255$
VARCHAR(M)	L+1个字节, 其中 $L \leq M$ 且 $0 \leq M \leq 65535$
TINYTEXT	L+1个字节, 其中 $L < 2^8$
TEXT	L+2个字节, 其中 $L < 2^{16}$
MEDIUMTEXT	L+3个字节, 其中 $L < 2^{24}$
LONGTEXT	L+4个字节, 其中 $L < 2^{32}$
ENUM('value1','value2',...)	1或2个字节, 取决于枚举值的个数(最多65,535个值)
SET('value1','value2',...)	1、2、3、4或者8个字节, 取决于set成员的数目(最多64个成员)

Value	CHAR (4)	Storage Required	VARCHAR (4)	Storage Required
' '	' '	4 bytes	' '	1 byte
'ab'	'ab '	4 bytes	'ab'	3 bytes
'abcd'	'abcd'	4 bytes	'abcd'	5 bytes
'abcdefgh'	'abcd'	4 bytes	'abcd'	5 bytes

日期型

列类型	存储范围	存储需求
TIME	-838:59:59 ~ 838:59:59	3
DATE	1000-01-01 ~ 9999-12-31	3
DATETIME	1000-01-01 00:00:00~ 9999-12-31 23:59:59	8
TIMESTAMP	1970-01-01 00:00:01 UTC ~ 2038-01-19 03:14:07	4
YEAR	1901~2155	1

日期型——DATETIME与TIMESTAMP的区别

相同

- 显示

TIMESTAMP列的显示格式与**DATETIME**列相同。换句话说，显示宽度固定在19字符，并且格式为YYYY-MM-DD HH:MM:SS。

不同

- 范围

DATETIME 以 ‘YYYY-MM-DD HH:MM:SS’ 格式检索和显示 **DATETIME**值。支持的范围为 ‘1000-01-01 00:00:00’ 到 ‘9999-12-31 23:59:59’，

TIMESTAMP值不能早于1970或晚于2037

日期型——DATETIME与TIMESTAMP的区别（续）

- 储存

TIMESTAMP 4个字节储存，存储空间比较小，表示的日期时间范围也比较小。底层存储方式不同，TIMESTAMP底层存储的是毫秒值，距离1970-1-1 0:0:0 0毫秒的毫秒值。**两个日期比较大小时，TIMESTAMP更方便、更快。**

DATETIME按8个字节储存，按实际格式储存。

- 时区

TIMESTAMP和时区有关。**TIMESTAMP**会根据用户的时区不同，显示不同的结果。

DATETIME与时区无关。只能反映出插入时当地的时区，其他时区的人查看数据必然会有误差的。

JSON类型

- ▶ JSON (JavaScript Object Notation) 是一种轻量级的 数据交换格式 。简洁和清晰的层次结构使得JSON 成为理想的**数据交换语言**。JSON数据可以在网络或者程序之间轻松地传递，并在需要的时候将它还原为各编程语言所支持的数据格式。在MySQL 5.7中，就已经支持JSON数据类型。

Creating JSON Values

A JSON array contains a list of values separated by commas and enclosed within [and] characters:

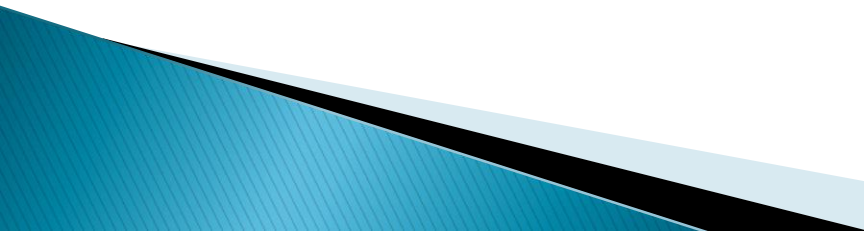
JSON 数组

```
["abc", 10, null, true, false]
```

A JSON object contains a set of key-value pairs separated by commas and enclosed within { and } characters:

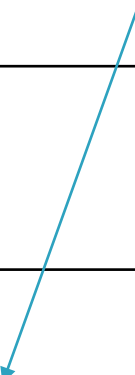
JSON 对象

```
{"k1": "value", "k2": 10}
```



例 员工表

Id	Num	name	Sex	birthday	address	salary
Smallint	char(10)	Varchar (20)	Enum (男, 女, 保密)	Date	JSON	DECIMAL(8,2)
0001	8133500 127	张三	男	1969-02- 22		8010. 75



`{"province": "江苏", "city": "无锡", "detail": "万科城市花园"}`

完整性约束

► 约束的功能—保证数据的完整性和一致性

1. MySQL的空值与非空
2. MySQL自动编号
3. MySQL主键约束
4. MySQL唯一约束
5. MySQL默认约束
6. MySQL的外键约束

► 约束的数目

1. 表级约束
2. 列级约束

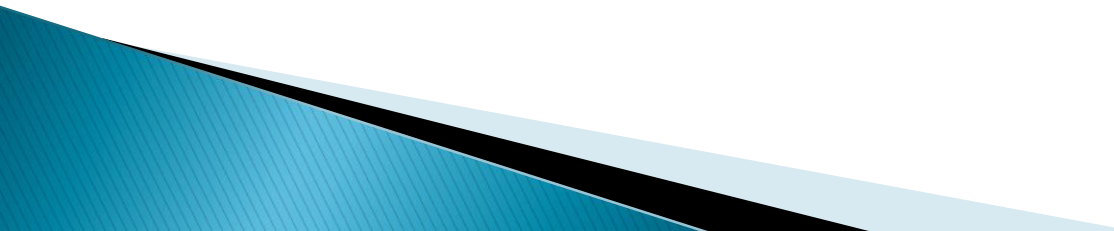
1 非空约束

- ▶ NULL，字段值可以为空
- ▶ NOT NULL，字段值禁止为空

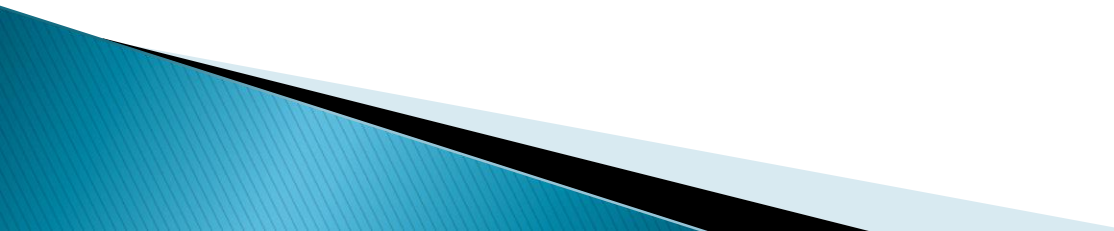
2 MySQL自动编号

- ▶ AUTO_INCREMENT
- ▶ 自动编号，**且必须与主键组合使用**
- ▶ 默认情况下，起始值为1，每次增量为1
- ▶ 设置自动编号的字段，其取值必须为整型

3 MySQL主键约束

- ▶ PRIMARY KEY
 - ▶ 每张数据表只能存在一个主键
 - ▶ 主键保证记录的唯一性
 - ▶ 主键自动为NOT NULL
- 

4 MySQL唯一约束

- ▶ UNIQUE
 - ▶ 唯一约束可以保证记录的唯一性
 - ▶ 唯一约束的字段可以为空值（NULL）
 - ▶ 每张数据表可以存在多个唯一约束
- 

5 MySQL默认约束

- ▶ DEFAULT
- ▶ 当插入记录时，如果没有明确为字段赋值，则自动赋予默认值

6 MySQL的外键约束

- ▶ FOREIGN KEY
- ▶ 保证数据一致性，完整性。
- ▶ 实现一对一或一对多关系。

创建表——MySQL的外键约束（续）

外键约束的要求：

- ▶ 父表和子表必须使用相同的存储引擎，而且禁止使用临时表。
- ▶ 数据表的存储引擎只能为InnoDB。
- ▶ 外键列和参照列必须具有相似的数据类型，其中数字的长度或是否有符号位必须相同；而字符的长度则可以不同。
- ▶ 外键列和参照列必须创建索引。如果外键列不存在索引的话，MySQL将自动创建索引。

例 员工表

Id	Num	name	Sex	birthday	address	salary
Smallint	char(10)	Varchar (20)	Enum (男, 女, 保密)	Date	JSON	DECIMAL(8,2)
0001	8133500 127	张三	男	1969-02- 22		8010. 75
主键自 增	工号唯一		非空 默认值保 密			



`{"province": "江苏", "city": "无锡", "detail": "万科城市花园"}`

查看当前数据库下已有数据表

- ▶ `SHOW TABLES;`
- ▶ `SHOW [FULL] TABLES [{FROM | IN} db_name]
[LIKE 'pattern' | WHERE expr]`

查看指定数据表的详细信息

- ▶ `SHOW CREATE TABLE tbl_name;`

查看表结构

- ▶ DESC tbl_name;
- ▶ DESCRIBE tbl_name;
- ▶ SHOW CREATE TABLE tbl_name;
- ▶ SHOW COLUMNS FROM tbl_name;

删除指定的数据表

- ▶ DROP TABLE [IF EXISTS] tbl_name;

表结构相关操作

▶ 添加字段

ALTER TABLE tbl_name

ADD 字段名称 字段属性 [完整性约束条件]
[FIRST|AFTER 字段名称]

▶ 删除字段

ALTER TABLE tbl_name

DROP 字段名称

表结构相关操作（续）

▶ 修改字段类型、字段属性

ALTER TABLE tbl_name

MODIFY 字段名称 字段类型 [字段属性] [FIRST | AFTER 字段名称]

▶ 修改字段名称、字段类型、字段属性

ALTER TABLE tbl_name

CHANGE 原字段名称 新字段名称 字段类型 字段属性 [FIRST | AFTER 字段名称]

表结构相关操作（续）

▶ 添加主键

```
ALTER TABLE tbl_name  
ADD PRIMARY KEY(字段名称)
```

▶ 删除主键

```
ALTER TABLE tbl_name  
DROP PRIMARY KEY;
```

表结构相关操作（续）

▶ 添加唯一

```
ALTER TABLE tbl_name
```

```
ADD UNIQUE KEY|INDEX [index_name] (字段名  
称)
```

▶ 删除唯一

```
ALTER TABLE tbl_name
```

```
DROP index_name;
```


表结构相关操作（续）

- ▶ 增加外键

```
ALTER TABLE tbl_name1
```

```
  ADD FOREIGN KEY(字段名) REFERENCES  
tbl_name2(字段名);
```

- ▶ 删除外键

```
ALTER TABLE tbl_name
```

```
DROP FOREIGN KEY 外键名;
```

表结构相关操作（续）

▶ 添加默认值

ALTER TABLE tbl_name

ALTER 字段名称 SET DEFAULT 默认值;

▶ 删除默认值

ALTER TABLE tbl_name

ALTER 字段名称 DROP DEFAULT

表结构相关操作（续）

▶ 修改数据表名称

```
ALTER TABLE tbl_name RENAME [TO|AS]  
new_tbl_name
```

```
RENAME TABLE tbl_name TO new_tbl_name;
```

▶ 修改存储引擎

```
ALTER TABLE tbl_name ENGINE=存储引擎名
```